

Université Sorbonne Paris Nord - B.U.T Métiers du Multimédia
et de l'Internet - Année 3 - Parcours Création Numérique



PROJET INTERACTIF SONORE

Création d'un jeu vidéo en Pixel Art sur
PlugData / GEM



Dossier de conception réalisé par JOAQUIM Alexy

Année Universitaire : 2025 - 2026

SOMMAIRE

1. DESCRIPTION DU PROJET	2
2. LES ÉLÉMENTS D'INTERFACES	2-3
3. DÉTAILS DES INTERACTIONS ET ORGANIGRAMME	3-13
3.1 - La Création et l'affichage du personnage	3-4
3.2 - Gestion des déplacements	4-5
3.3 - Gestion du saut	6
3.4 - Création des éléments interactifs	7-8
3.5 - Interactions et collisions	8-13
4. LES DIFFICULTÉS RENCONTRÉES	13-15
4.1 - La gestion des collisions	13
4.2 - La gestion du saut du personnage	13
4.3 - Tentative de mise en place d'un super saut	14
4.4 - Interactions avec la case mobile	14
4.5 - Débogage et vérification des connexions	15
5. MES RETOURS RÉFLÉXIFS	15
6. LES AXES D'AMÉLIORATIONS SI PLUS DE TEMPS	16

1. DESCRIPTION DU PROJET

Ce projet consiste en la création d'un mini jeu interactif en Pixel Art réalisé avec PlugData et la bibliothèque GEM. Le joueur contrôle un personnage pouvant se déplacer horizontalement et sauter dans un environnement composé de plateformes fixes, mobiles et de cases interactives.

Le projet intègre plusieurs fonctionnalités :

- la gestion des déplacements du personnage
- le système de collisions
- les plateformes dynamiques (case mouvante, case animée)
- le déclenchement d'événements sonores
- le changement d'apparence des cases lors des collisions

L'objectif principal de ce projet était de concevoir une expérience interactive en temps réel et de découvrir une grande partie des possibilités de création offertes par PlugData.

2. LES ÉLÉMENTS D'INTERFACES

L'interface du projet est composée de plusieurs éléments visuels et interactifs, pour permettre au joueur d'évoluer dans l'environnement du jeu (voir image si-dessous) :



Comme nous pouvons le voir sur l'image, nous retrouvons notamment :

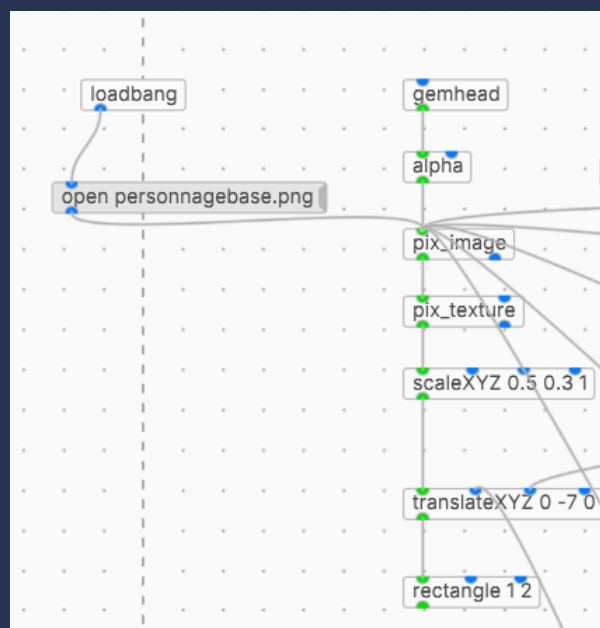
- un personnage principal contrôlable au clavier
- un décors en Pixel Art servant d'arrière-plan
- des plateformes fixes et mobiles
- des cases mystères interactives
- des éléments sonores déclenchés lors de certaines collisions

Le déplacement du personnage s'effectue horizontalement à l'aide du clavier, tandis que le saut permet d'interagir avec les différentes plateformes présentes dans la scène.

L'ensemble des éléments graphiques a été intégré dans PlugData à l'aide de GEM afin de créer un environnement interactif simple inspiré des jeux de plateformes rétro, notamment Mario Bros.

3. DÉTAILS DES INTERACTIONS ET ORGANIGRAMME

3.1 - La Création et l'affichage du personnage



Arborescence de l'initialisation du personnage

Comme nous pouvons le voir sur l'image du dessus, le personnage du jeu a été créé à partir d'une image en Pixel Art importée dans PlugData grâce à l'objet `pix_image`.

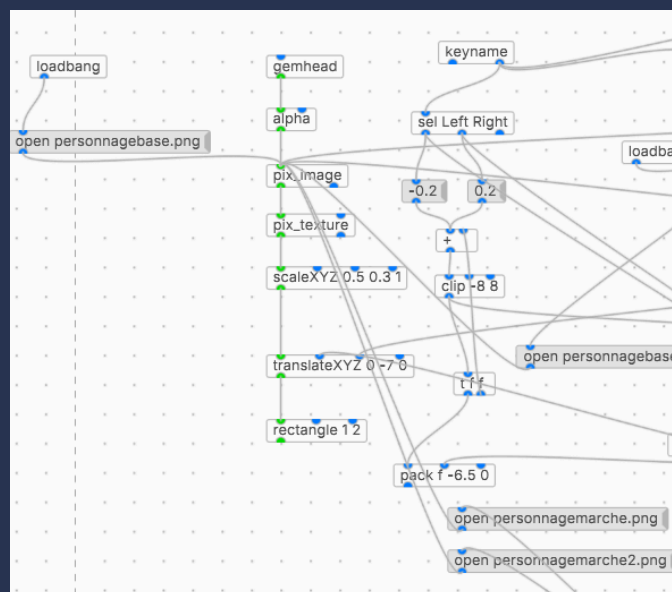
L'affichage du personnage dans la scène est géré par plusieurs objets de GEM :

- **gemhead** qui permet d'intégrer l'élément dans la scène graphique
- **pix_texture** qui applique l'image sur une texture
- **translateXYZ** qui définit la position du personnage dans l'espace
- **rectangle** qui affiche l'image à l'écran, dans une dimension XYZ

Le personnage possède une position initiale située sur l'axe X et Y de la scène afin d'être placé directement dans le décor lors du lancement du projet.

Pour le faire déplacer, des sprites différents ont également été utilisés pour créer une animation de déplacement et de saut.

3.2 - Gestion des déplacements



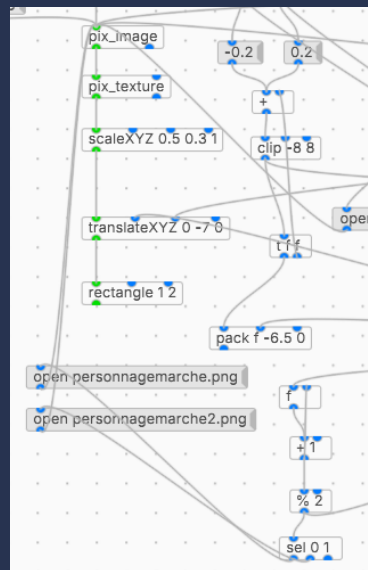
Le déplacement du personnage est contrôlé grâce à l'objet **keyname**, qui permet de détecter les touches directionnelles du clavier utilisées par le joueur.

Les touches gauche et droite sont ensuite traitées par l'objet **sel Left Right**, qui différencie les directions afin d'appliquer un déplacement positif (ici + 0.2) ou négatif (-0.2) sur l'axe des X.

Le système utilise également l'objet `clip` afin de limiter les déplacements du personnage et l'empêcher de sortir de la scène.

La position du personnage est ensuite mise à jour dans l'espace grâce à `translateXYZ`, qui modifie ses coordonnées en temps réel dans l'environnement GEM.

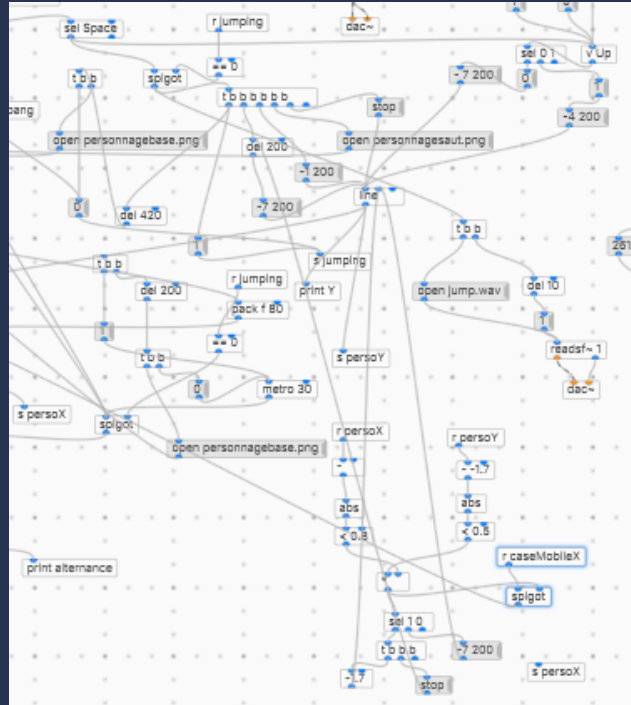
Afin de rendre les déplacements du personnage plus dynamiques, deux images différentes de marche ont été utilisés pour créer une animation de marche :



Le système fonctionne grâce à différents messages open, reliés à l'objet `pix_image`, qui permet de charger automatiquement plusieurs images du personnage pendant les déplacements.

L'objet `sel` et les différents déclencheurs permettent d'alterner entre les images `personnagemarche.png` et `personnagemarche2.png` pour donner une impression de mouvement lorsque le personnage se déplace dans le décor. Ensuite, les images sont appliquées dans la scène grâce à l'objet `pix_texture`.

Le schéma si-dessus présente une arborescence plus complexe. Mais en réalité, le fonctionnement est plus simple.

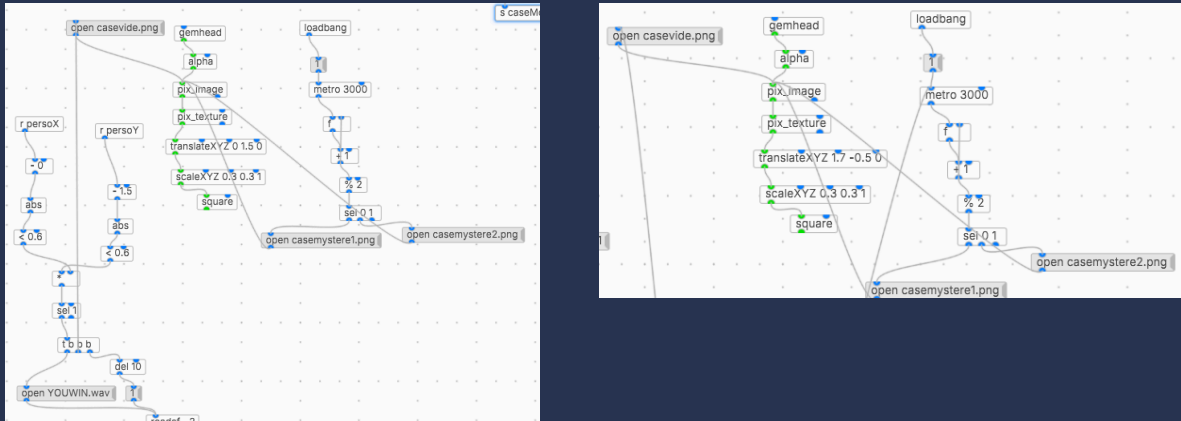


Le saut du personnage est déclenchée grâce à la touche `Space`, détectée avec les objets `keyname` et `sel Space`. Lorsqu'un saut est activé, un premier message modifie la position verticale du personnage sur l'axe Y pour créer une montée (saut à Y augmentée à -1 en 200ms). Après un court délai généré par `del`, un second message ramène progressivement le personnage vers le sol (-7 en 200ms). Le mouvement vertical est rendu plus fluide grâce à l'objet `line`, qui permet de créer une transition progressive entre les différentes positions du personnage. Le système utilise également un changement d'image temporaire, avec `open personnagesaut.png`, qui affiche une image spécifique du personnage pendant le saut. Un effet sonore `jump.wav` a également été ajouté afin de rendre l'expérience de jeu plus enrichissante.

Enfin, l'objet `spigot` a été utilisé pour contrôler certains déclenchements et éviter que plusieurs actions ne se produisent simultanément pendant le saut.

3.4 - Création des éléments interactifs

Plusieurs éléments interactifs ont été intégrés dans l'environnement du jeu afin de rendre l'expérience plus dynamique :



L'arborescence des deux cases mystères

Tout comme le personnage, les cases mystères utilisent un système d'alternance automatique d'images pour créer une animation d'effet visuel dynamique. Grâce à l'objet **metro 3000**, un déclenchement automatique est effectué toutes les 3 secondes. Les objets **sel** et les différents messages **open casemystere1.png** et **open casemystere2.png** permettent ainsi d'alterner automatiquement entre plusieurs images.

casemystere1.png

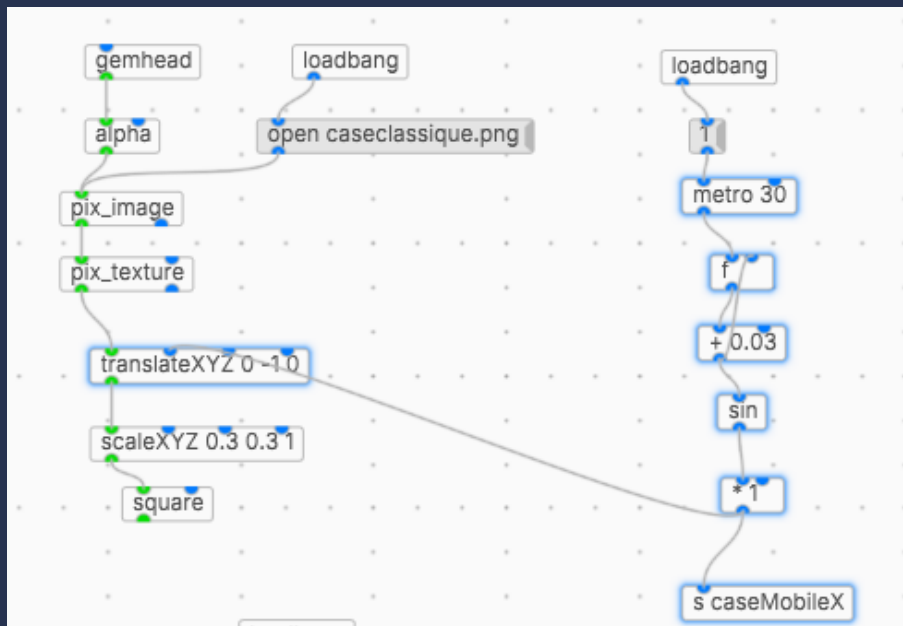


casemystere2.png



Une case mobile a également été intégrée dans le projet pour rendre l'environnement plus dynamique.

Arborescence de la case classique mouvante



Le déplacement automatique de la plateforme est généré grâce à l'objet **metro 30**, qui envoie des impulsions régulières dans le patch. Ces impulsions alimentent plusieurs opérations mathématiques ainsi qu'une fonction sin, qui permet de produire un mouvement horizontal continu.

Enfin, la valeur de déplacement calculée est stockée avec **s caseMobileX**, qui met à jour la position de la plateforme en temps réel dans la scène interactive.

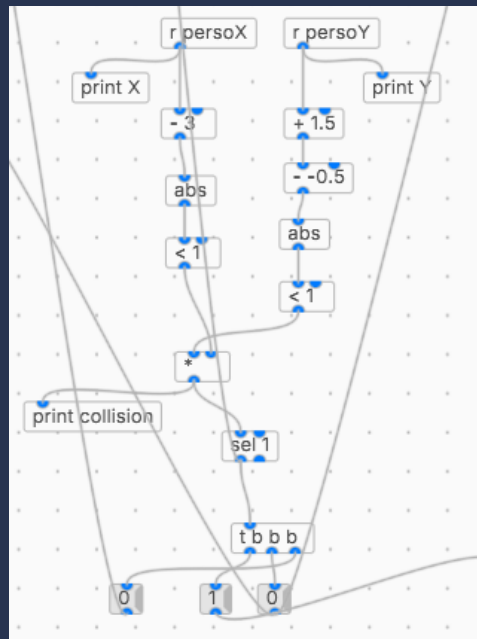
La méthode d'importation de l'image de la case mouvante reste la même que celui du personnage et des cases mystères.

3.5 - Interactions et collisions

3.5.1 - Détection des collisions

La détection des collisions repose sur la comparaison des coordonnées du personnage avec celles des éléments interactifs présents dans la scène.

Arborescence de la détection des collisions



Comme indiqué dans l'arborescence, les objets `r persoX` et `r persoY` permettent de récupérer les positions du personnage sur les axes X et Y. Les opérations `abs` et `< 1` permettent ensuite de vérifier si le personnage se trouve suffisamment proche d'un objet pour déclencher une collision.

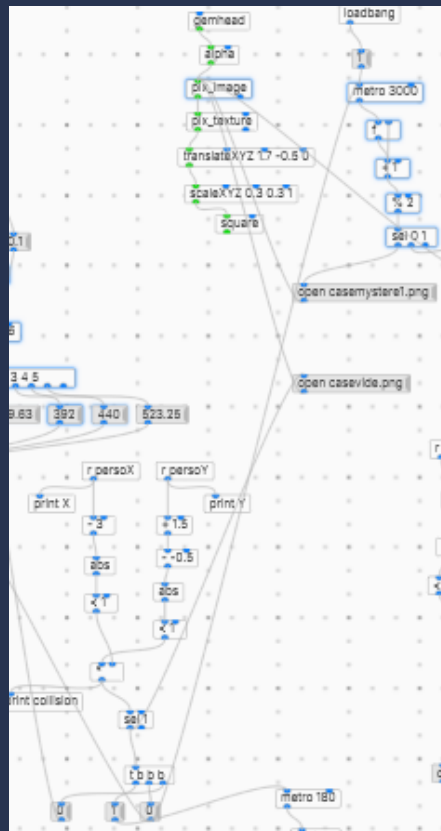
Si les deux conditions sont validées simultanément, l'objet `sel 1` active les différentes interactions du projet, comme le déclenchements des sons ou le changement visuel des cases mystères. Exemple :

case mystère avant la collision



case mystère après la collision



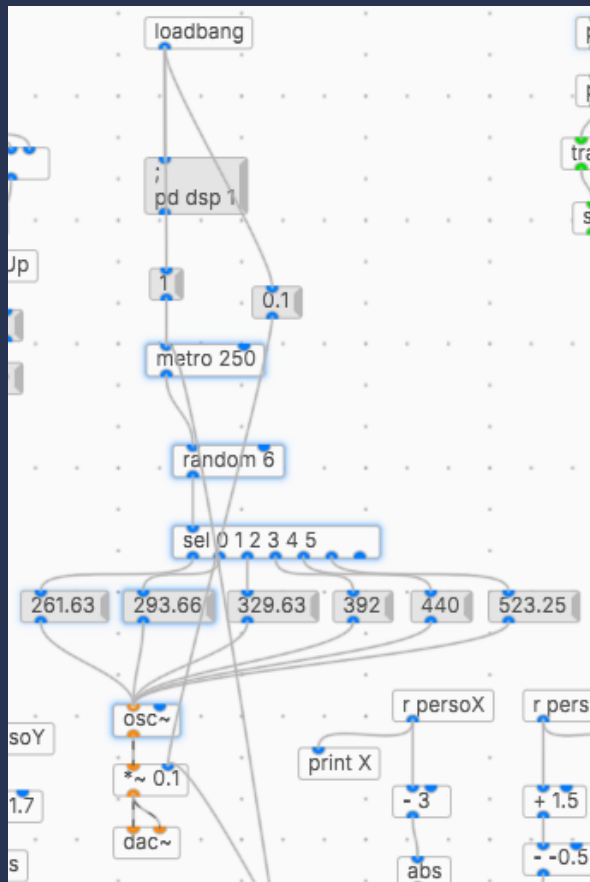


D'après cet arborescence, lorsqu'une collision entre le personnage et la case est détectée grâce aux objets `abs`, `< 1`, `*` et `sel 1`, un nouveau message `open casevide.png` (relié entre le `sel 1` et `pix_image` de la case) est déclenché, qui va remplacer l'image de la case mystère par une case vide.

Ces cases mystères permettent également de déclencher un son ou une musique lorsqu'elle entre en collision avec le personnage.

3.5.2 - Interactions sonores

Lors du lancement du Plug, la musique initiale du jeu-vidéo est lancé automatiquement à l'aide de l'objet `loadbang`, présent dans l'arborescence si dessous :

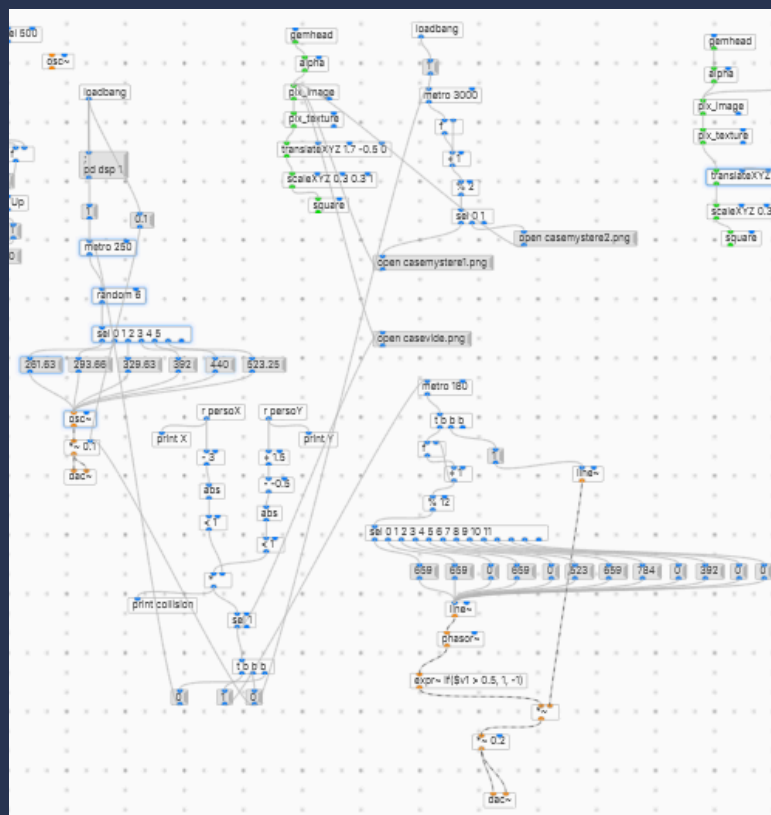


Arborescence de la musique initiale

Ce système sonore a été intégré au projet pour créer une ambiance audio dynamique dès le lancement du jeu. L'objet **metro 250** envoie des impulsions régulières qui activent ensuite l'objet **random 6**, permettant de sélectionner des valeurs aléatoires parmi les 6 présentes (toutes les 250 ms). L'objet **sel 0 1 2 3 4 5** répartit ensuite ces valeurs vers plusieurs fréquences différentes pour créer des variations sonores automatiques.

Ces fréquences sont envoyées à l'objet **osc~**, qui génère les sons. Puis, le signal audio est transmis vers les deux entrées de **dac~** pour que le son soit diffusé dans le projet interactif.

Les cases mystères peuvent également déclencher un son en plus de changer de forme lorsqu'elles entrent en collision avec le personnage. En voici un exemple de l'arborescence :

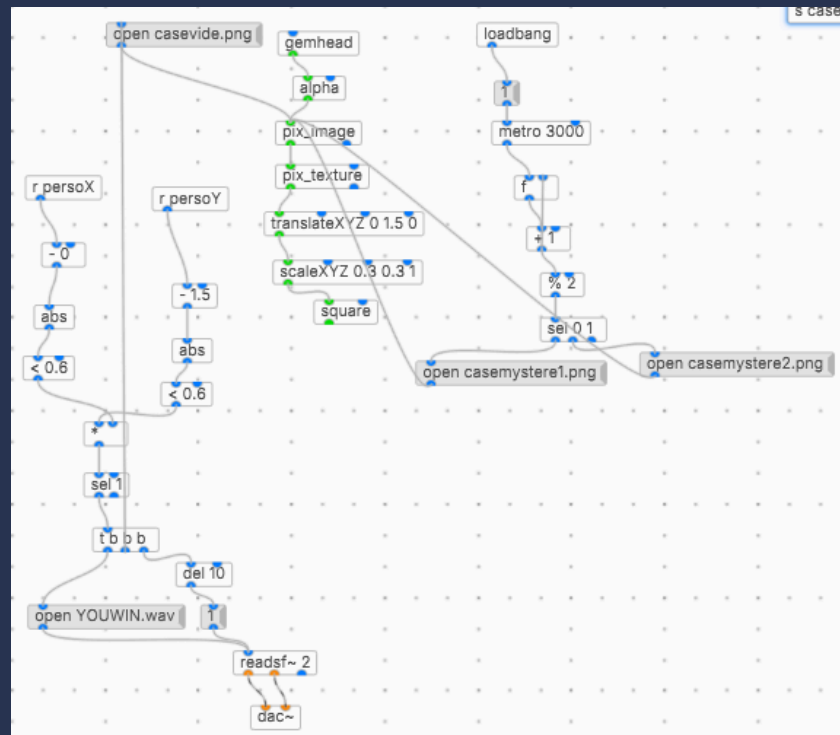


Cette arborescence présente l'intégralité du fonctionnement des interactions visuelles et sonores du projet. Comme mentionné précédemment, les objets `r_persoX` et `r_persoY` récupèrent les coordonnées du personnage. Lorsqu'une collision est détectée entre le personnage et cette case mystère, plusieurs actions sont déclenchées :

- Le changement d'apparence de la case mystère en case vide
- Le déclenchement d'une nouvelle ambiance sonore grâce au système audio relié au système de détection des coordonnées du personnage et dans le système de la case mystère.

Les objets `metro`, `random`, `osc~`, `phasor~`, `line~` et `dac~` permettent ensuite de générer et de diffuser les différents sons du projet.

L'arborescence de la 2e case mystère reste sur le même principe pour le déclenchement de la musique que la première, mais un son YOUWIN.wav est importé directement pour la lancer si le personnage arrive à toucher la case :



4. LES DIFFICULTÉS RENCONTRÉES

4.1 - La gestion des collisions

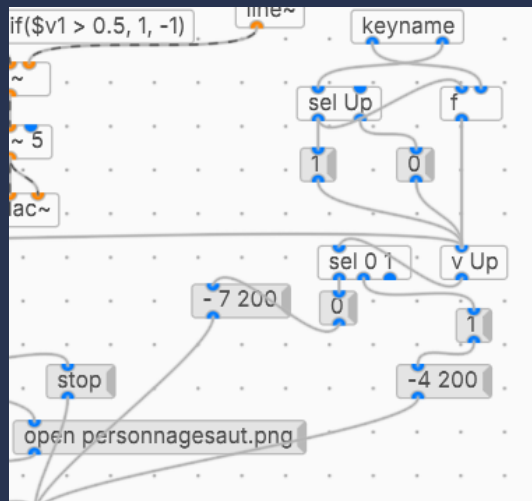
La mise en place du système de collision a constitué l'une des principales difficultés du projet. Il a été nécessaire de comparer les coordonnées du personnage avec celles des différents objets interactifs afin de détecter correctement les contacts. Plusieurs ajustements des valeurs de comparaison ont été réalisés pour obtenir un résultat satisfaisant.

4.2 - La gestion du saut du personnage

La création du système de saut a demandé plusieurs essais afin d'obtenir un mouvement cohérent. Le réglage des objets `line` et `del` a nécessité de nombreux tests pour obtenir une montée et une descente fluides du personnage.

4.3 - Tentative de mise en place d'un super saut

Une tentative de super saut a été mise en place en combinant la touche Espace avec la flèche du haut. Le système reposait sur la détection de **Up** avec **keyname**, le stockage de l'état dans **v Up**, puis le choix entre un saut normal et un saut plus haut avec **sel 0** **1**. Cependant, le système s'est révélé plus complexe que prévu et n'a pas pu être finalisé dans le temps imparti. En voici un extrait :



4.4 - Interactions avec la case mobile

Une autre difficulté concernait l'interaction entre le personnage et la plateforme mobile. L'objectif était que le personnage reste correctement positionné sur la plateforme lorsqu'elle se déplace. Malgré plusieurs tentatives de synchronisation à l'aide de la variable **caseMobileX**, cette fonctionnalité n'a pas pu être entièrement finalisée et le personnage ne peut pas tomber de la case mouvante et ne suit pas son mouvement.



4.5 - Débogage et vérification des connexions

Tout au long du projet, plusieurs phases et débogage ont été nécessaires, voire primordial pour vérifier le bon fonctionnement des différentes connexions du patch. Une attention particulière a été portée aux entrées et sorties des objets, ainsi qu'à la circulation des messages entre les différents systèmes (déplacements, sauts, collisions et interactions) pour corriger plusieurs erreurs de logique et de connexion au cours de la réalisation.

5. MES RETOURS RÉFLÉXIFS

Ce projet m'a permis de découvrir plus en profondeur le fonctionnement de PlugData et de la programmation nodale. Au début du projet, la logique des connexions entre les objets pouvait sembler complexe, mais les différentes phases de développement et de débogage m'ont progressivement permis de mieux comprendre le fonctionnement du logiciel.

La réalisation de ce projet m'a également appris l'importance de la méthode et de l'organisation lors de la création d'un patch de grande taille. Au fil du développement, certaines fonctionnalités sont devenues plus difficiles à modifier ou à corriger en raison du nombre important de connexions présentes dans le projet.

Malgré plusieurs difficultés techniques rencontrées, notamment lors de la mise en place des collisions, du saut ou de la plateforme mobile, le projet m'a permis d'acquérir une meilleure compréhension des systèmes interactifs en temps réel et de développer ma capacité à résoudre des problèmes techniques. Certaines pistes d'amélioration restent néanmoins envisageables afin d'enrichir davantage l'expérience proposée.

6. LES AXES D'AMÉLIORATIONS SI PLUS DE TEMPS

Avec davantage de temps, le projet aurait pu évoluer vers un véritable jeu de plateforme inspiré de licences comme Super Mario Bros.

L'environnement actuel aurait pu être étendu à une carte beaucoup plus vaste, avec un défilement horizontal permettant au joueur d'explorer un niveau complet plutôt qu'une seule scène fixe. De nouveaux obstacles, plateformes et zones interactives auraient également pu être ajoutés afin de proposer une expérience de jeu plus riche.

Les mécaniques de déplacement auraient pu être approfondies grâce à l'ajout d'un double saut et d'un contrôle de personnage pendant le saut.

Les cases mystères auraient également pu jouer un rôle plus important dans le gameplay en accordant des bonus au joueur, comme des améliorations temporaires, des points supplémentaires ou des effets spéciaux, en plus de déclencher des interactions sonores.

L'ajout d'un système de points de vie, d'ennemis, d'objectifs et de progression aurait permis de transformer ce prototype interactif en un véritable jeu d'aventure doté de mécaniques plus complètes.

Un système de score ou de collecte d'objets aurait également pu être mis en place afin d'encourager l'exploration du niveau et de donner davantage d'objectifs au joueur.